PUMS ↔ AutoCount Integration — API Specification

Audience: PUM System Developer

This document defines the API contracts between PUMS and the AutoCount integration. There are **four endpoints across two directions**:

Name	Endpoint	Hosted by	Direction	Method	Machine Scope
API 1	`/api/meter-reading/online`	**PUMS**	AutoCount → PUMS	`GET`	ONLINE machines
API 2	`/api/meter-reading/offline`	**PUMS**	AutoCount → PUMS	`GET`	OFFLINE machines
Webhook 1 — Stock Issue Request	`/api/stockissue`	**AutoCount webhook —** `https://atpwebhook.prismatechnology.com.my`	PUMS → AutoCount	`POST`	—
Webhook 2 — Stock Transfer Request	`/api/stocktransfer`	**AutoCount webhook —** `https://atpwebhook.prismatechnology.com.my`	PUMS → AutoCount	`POST`	—

API 1 and **API 2** are implemented **by PUMS** (AutoCount calls them). **Webhook 1** and **Webhook 2** are consumed **by PUMS** (PUMS calls the AutoCount webhook).

API 1 — Meter Reading (Machine Status: ONLINE)

Scope: Only machines with `Machine Status = ONLINE` are in scope for this API. Reference: https://atgroup.asia/all_device/ — filter the device list to ONLINE machines only. Machines that are not ONLINE must be omitted from the response.

Implemented by: PUMS
Method: `GET`
Path: `/api/meter-reading/online`

Returns the **latest** BK (Black) and CL (Color) meter readings for **all** ONLINE machines. No request body and no query parameters — AutoCount fetches the full list in one call.

Request

No request body. No query parameters. AutoCount simply issues `GET /api/meter-reading/online` with the standard `X-API-Key` header.

Response — `200 OK`

JSON array. One object per ONLINE machine. Machines that are not ONLINE are omitted.

For each machine, return the **latest** meter reading available (most recent audit), regardless of date.

Field	Type	Sample Value	Description
`Code`	`string`	`CSSI 00002153`	Machine code.
`SerialNumber`	`string`	`XZL02175`	Machine serial number.
`TotalBK`	`integer`	`17257`	Latest Total Black meter reading.
`TotalCL`	`integer`	`11978`	Latest Total Color meter reading.
`LastAuditDate`	`string` (ISO-8601)	`2026-05-12T15:01:32`	Date/time of the latest audit reading.

Example

Request

```
GET /api/meter-reading/online
X-API-Key: <your-api-key>
```

Response

```json
[
  {
    "Code": "CSSI 00002153",
    "SerialNumber": "XZL02175",
    "TotalBK": 17257,
    "TotalCL": 11978,
    "LastAuditDate": "2026-05-12T15:01:32"
  },
  {
    "Code": "CSSI 00000090",
    "SerialNumber": "JBA05089",
    "TotalBK": 1562075,
    "TotalCL": 41833,
    "LastAuditDate": "2026-05-13T14:58:05"
  }
]
```

API 2 — Meter Reading (Machine Status: OFFLINE)

Scope: Only machines with `Machine Status = OFFLINE` are in scope for this API. Reference: https://atgroup.asia/tasks_meter/ — the readings come from the **Tasks Meter** report, which is how OFFLINE machines have their meters captured.

Implemented by: PUMS
Method: `GET`
Path: `/api/meter-reading/offline`

Returns the **latest** BK (Black) and CL (Color) meter readings for OFFLINE machines whose meter task was **created within the requested month**, plus the Task Tracking ID. AutoCount supplies only the `month` and PUMS returns the full set for that month — no per-machine filtering on the AutoCount side.

Request — Query String

Param	Type	Required	Sample	Description
`month`	`int` (1–12)	Yes	`5`	Billing month. PUMS filters tasks by **task create date** falling inside this month.

Filter rules (applied by PUMS)

1. **Date filter:** task **create date** is inside the requested month.
2. **Type filter:** `TrackingId` **contains** `MR` — this includes both `PM/MR` and `MR` tasks, and excludes `SN` tasks.
3. **Reading filter:** the meter column on the task is **filled** — at least one of `TotalBK` / `TotalCL` must be present **and greater than 0**. Tasks where **both** BK and CL are empty or `0` are excluded.
4. **De-dup by SerialNumber:** if a single machine has more than one qualifying task in the month (e.g. a `PM/MR` on `2026-05-05` and an `MR` on `2026-05-25`), return **only the latest** (by create date) — one row per `SerialNumber` per month.

Case-study sanity check: April 2026 has 5000 tasks total → 1000 `SN`, 2000 `PM/MR`, 1000 `MR`. After the filter (steps 2–4) the API returns **3000** rows = 2000 `PM/MR` + 1000 `MR`, deduped to the latest per serial.

Response — `200 OK`

JSON array. One object **per OFFLINE machine** that has a qualifying task in the requested month. Machines without a qualifying task are simply omitted (do **not** return them with `TrackingId = null`).

Field	Type	Sample Value	Description
`Code`	`string`	`CSSI 00002153`	Machine code.
`SerialNumber`	`string`	`XZL02175`	Machine serial number. Unique within the response.
`TotalBK`	`integer`	`17257`	Latest Total Black meter reading from the qualifying task.
`TotalCL`	`integer`	`11978`	Latest Total Color meter reading from the qualifying task.

Field	Type	Sample Value	Description
`LastAuditDate`	`string` (ISO-8601)	`2026-05-25T15:01:32`	Task **create date** of the qualifying task (the one returned).
`TrackingId`	`string` (**required, non-null, contains** `MR`)	`MR-2026-0525-008`	Tracking ID of the qualifying task. **Always present**, and always contains `MR` (either `PM/MR-…` or `MR-…`).

Example

Request

```
GET /api/meter-reading/offline?month=5
X-API-Key: <your-api-key>
```

Response

```
[
  {
    "Code": "CSSI 00002153",
    "SerialNumber": "XZL02175",
    "TotalBK": 17257,
    "TotalCL": 11978,
    "LastAuditDate": "2026-05-25T15:01:32",
    "TrackingId": "MR-2026-0525-008"
  },
  {
    "Code": "CSSI 00000090",
    "SerialNumber": "JBA05089",
    "TotalBK": 1562075,
    "TotalCL": 41833,
    "LastAuditDate": "2026-05-13T14:58:05",
    "TrackingId": "PM/MR-2026-0513-014"
  }
]
```

Webhook 1 — Stock Issue Request (PUMS → AutoCount)

Called by: PUMS
Method: `POST`
URL: `https://atpwebhook.prismatechnology.com.my/api/stockissue`
Content-Type: `application/json`
Reference: https://atgroup.asia/stockReport/ — source of the Stock Issue records on the PUMS side.

PUMS pushes **one** Stock Issue record to AutoCount per call.

Request Body — JSON object

Field	Type	Description
`StockIssueId`	`string`	**Unique ID generated by PUMS.** Used for idempotency.
`IssueDateTime`	`string` (ISO-8601)	Issue date/time.
`StockIssueNo`	`string`	Stock Issue document number.
`ReferenceNo`	`string`	Reference number.
`Description`	`string`	Description.
`Department`	`string`	Department code.
`Job`	`string`	Job code.
`Technician`	`string`	Technician code / name.
`Location`	`string`	Stock location code.
`ItemCode`	`string`	Item code.
`Quantity`	`number`	Quantity issued.
`UOM`	`string`	Unit of measure.

```json
{
  "StockIssueId":   "PUMS-SI-000123",
  "IssueDateTime": "2026-01-01T00:00:00",
  "StockIssueNo":   "DOS2601/001",
  "ReferenceNo":    "RPS-251230-043",
  "Description":    "Stock Issue-REDHA",
  "Department":     "A/4PE20147",
  "Job":            "MC001",
  "Technician":     "AUTOCOUNT",
  "Location":       "PERLING",
  "ItemCode":       "NPG89 T/BK",
  "Quantity":       1,
  "UOM":            "PCS"
}
```

Response

Success — `200 OK`

```json
{
  "success": true,
  "stockIssueId": "PUMS-SI-000123",
  "message": "Stock Issue task received."
}
```

Failure — non-2xx (e.g. `400`, `409`, `422`, `500`)


```
{
  "success": false,
  "stockIssueId": "PUMS-SI-000123",
  "errorCode": "INVALID_PAYLOAD",
  "reason": "Field 'Quantity' is required."
}
```


Field	Type	Description
`success`	`boolean`	`true` on accept, `false` on reject.
`stockIssueId`	`string`	Echo of the request `StockIssueId`.
`message`	`string`	Human-readable success note (present only when `success = true`).
`errorCode`	`string`	Machine-readable error code (see table below). Present only when `success = false`.
`reason`	`string`	Human-readable explanation of the failure. Present only when `success = false`.

Error Codes

HTTP Status	`errorCode`	Meaning
`400`	`INVALID_PAYLOAD`	Required field missing or wrong type.
`500`	`INTERNAL_ERROR`	Unexpected server error. Retry recommended.

> **Duplicates are allowed.** Re-sending the same `StockIssueId` is **not** an error — the previously received payload for that ID is overwritten with the new one. PUMS may safely re-submit without checking whether a `StockIssueId` has been sent before. (Note: this endpoint only **receives** the data; it does not auto-create an AutoCount Stock Issue document. Document creation is handled separately on the AutoCount side.)

Webhook 2 — Stock Transfer Request (PUMS → AutoCount)

Called by: PUMS
Method: `POST`
URL: `https://atpwebhook.prismatechnology.com.my/api/stocktransfer`
Content-Type: `application/json`
Reference: https://atgroup.asia/stockStandby/ — source of the Stock Transfer records on the PUMS side.

PUMS pushes **one** Stock Transfer record to AutoCount per call.

Request Body — JSON object

Field	Type	Description
`RequestId`	`string`	**Unique ID generated by PUMS.** Used for idempotency.
`DocumentDateTime`	`string` (ISO-8601)	Document date/time.
`Technician`	`string`	Technician code.
`Part`	`string`	Part / item code.
`qty`	`number`	Quantity.
`type`	`string`	Transfer type.
`unit`	`string`	Unit of measure.
`approval`	`string`	Approval reference.


```
{
  "RequestId":        "50460",
  "DocumentDateTime": "2026-05-06T11:18:09",
  "Technician":       "AUTOCOUNT",
  "Part":             "TC001 [USED PART]",
  "qty":              -2,
  "type":             "OUT",
  "unit":             "UNIT",
  "approval":         "Yes"
}
```


Response

Success — `200 OK`


```
{
  "success": true,
  "requestId": "50460",
  "message": "Stock Transfer task received."
}
```


Failure — non-2xx (e.g. `400`, `500`**)**


```
{
  "success": false,
  "requestId": "50460",
  "errorCode": "INVALID_PAYLOAD",
  "reason": "Field 'qty' is required."
}
```

Field	Type	Description
`success`	`boolean`	`true` on accept, `false` on reject.
`requestId`	`string`	Echo of the request `RequestId`.
`message`	`string`	Human-readable success note (present only when `success = true`).
`errorCode`	`string`	Machine-readable error code (see table below). Present only when `success = false`.
`reason`	`string`	Human-readable explanation of the failure. Present only when `success = false`.

Error Codes

HTTP Status	`errorCode`	Meaning
`400`	`INVALID_PAYLOAD`	Required field missing or wrong type.
`500`	`INTERNAL_ERROR`	Unexpected server error. Retry recommended.

> **Master data and field values are not validated.** Unknown `Part`, `Tech`, or `type` values are accepted as-is — AutoCount will not reject the document on those grounds. Make sure PUMS sends correct values; AutoCount won't tell you if they don't match.

> **Duplicates are allowed.** Re-sending the same `RequestId` is **not** an error — the previously received payload for that ID is overwritten with the new one. PUMS may safely re-submit without checking whether a `RequestId` has been sent before. (Note: this endpoint only **receives** the data; it does not auto-create an AutoCount Stock Transfer document. Document creation is handled separately on the AutoCount side.)

Conventions

- **Encoding:** `UTF-8`. All bodies are JSON.
- **Timestamps:** ISO-8601, preferably UTC (`2026-03-12T08:45:00Z`).
- **One record per call:** Webhook 1 and Webhook 2 each accept a **single** JSON object per request (not an array). PUMS sends one HTTP call per Stock Issue / Stock Transfer.
- **Upsert semantics (duplicates allowed):** `StockIssueId` (Webhook 1) and `RequestId` (Webhook 2) act as the key for the received payload. Re-sending the same ID overwrites the previously received payload — it does **not** auto-create or modify any AutoCount document. Document creation is handled separately on the AutoCount side. PUMS does not need to track which IDs have been sent.
- **Response shape:**
    - On success → HTTP `2xx` with `{ "success": true, ... }`.
    - On failure → HTTP non-2xx with `{ "success": false, "errorCode": "...", "reason": "..." }`.
- **Retries:**

- HTTP `5xx` (e.g. `INTERNAL_ERROR`) or transport failure → retry with exponential backoff (safe — re-send is an upsert).
- HTTP `4xx` (e.g. `INVALID_PAYLOAD`) → do **not** retry blindly; the payload must be corrected first.

- **Authentication: API key** — all four endpoints (API 1, API 2, Webhook 1, Webhook 2) require an API key sent in the HTTP header `X-API-Key`. Keys are issued per integration partner and must be kept secret. Requests without a valid key return `401 Unauthorized`.

```
X-API-Key: <your-api-key>
```

Open Items

- API key exchange — PUMS and AutoCount sides each issue a key for the other to use.
- Field length / numeric precision limits.
- Base URL for API 1 and API 2 (both PUMS-hosted). The relative paths are defined in this document; only the host needs to be supplied.
- Final list of `errorCode` values (the tables above are a starting set).